§ Contents

CrisisGrid

An AI-assisted emergency operations platform connecting citizens, dispatchers, rescue teams, hospitals, and shelters on one real-time map — built on the MERN stack.

MongoDB | Express.js | React | Node.js | Socket.IO | Google Gemini (free tier) | Leaflet / OpenStreetMap | Tailwind CSS

This document is written for two audiences at once: anyone evaluating the project (recruiter, interviewer, visitor) who wants to understand what it does and how to use it, and anyone reading the code who wants a map of how each feature is implemented before diving in.

1 Project Overview & Vision

CrisisGrid is a unified emergency-operations platform that connects everyone involved in responding to a disaster — citizens reporting what they see, an Emergency Operations Center (EOC) coordinating the response, rescue teams executing it, and hospitals/shelters tracking their own capacity — on one shared, real-time picture of the situation.

The core problem it solves: during a disaster, information is scattered across phone calls, radio chatter, and word of mouth. CrisisGrid puts every report on one live map, automatically triages and verifies it, and pushes updates to every relevant screen the instant something changes.

Who uses it

ROLE	WHAT THEY DO ON THE PLATFORM
Citizen	Sends SOS alerts and incident reports, sees nearby shelters/hospitals, gets safety guidance from the assistant chatbot.
Emergency Operations Center (EOC)	Sees every incident on a live dashboard, verifies or rejects reports, dispatches resources, broadcasts alerts, reviews organization documents.
Rescue Team	Sees assigned jobs and unassigned high-priority incidents, self-assigns, updates status through to resolution.
Hospital	Registers their facility, pushes live bed-availability updates that appear instantly on every map.
Shelter	Same as Hospital, but tracks occupancy/capacity instead of beds.

Design principle: honestly scoped, not superficially broad

The original concept included 8 user roles, offline-first sync, background job queues, six disaster scenarios, and a donations module. Rather than half-build all of it, this build implements five roles and a focused feature set *completely* — every feature described in this document is real, working, and demoable end-to-end, not a mockup. Section 6 lists what was deliberately deferred and why.

2 User Roles & How to Navigate the Site

Getting in

1. Open the site — you land on the public homepage with a short pitch and role overview.
2. Click **Get started** to register, or **Sign in** if you already have an account.
3. On the login page, demo accounts are one click away — quick-fill buttons for Citizen, EOC, and Rescue Team autocomplete the demo credentials.
4. Registering as Hospital, Shelter, Rescue Team, NGO, or Volunteer prompts an extra step: uploading a verification document (any image works for the demo — see Section 3).
5. After login, you're routed automatically to the dashboard for your role — you never see a menu of dashboards you don't have access to.

Citizen dashboard

A live map of nearby incidents and facilities, a list of your own past reports with live status, and a prominent `+ Report incident / SOS` button. The report form has two modes — SOS (immediate danger) or a standard report — and shows a live AI preview of how the report will be classified before you even submit it.

EOC (Emergency Operations Center) dashboard

The command center: an operational map, a live incident feed with severity filters, an AI-generated situation summary banner, an escalated-incidents panel, an emergency contact directory (with document approval controls), and an alert broadcast panel. Every incident card has one-click status advancement and verification controls.

Rescue team console

Two lists — "your assignments" and "unassigned & urgent" — plus the live map. Self-assign any unassigned high-priority incident with one click; mark your own jobs resolved when done.

Hospital / Shelter portal

First-time login prompts a short facility registration form (name, address, total capacity, contact, resources). After that, a capacity slider and status selector push live updates to every dashboard's map instantly.

Always available: the assistant chatbot

A floating chat button, bottom-right, on every page. For citizens it gives safety guidance and the nearest shelter; for EOC/rescue it answers natural-language questions like "show all unverified flood reports" with a live, clickable result list.

> **Tip for evaluators:** the fastest way to see the whole platform is to open two browser windows — sign in as `citizen@demo.crisisgrid.app` in one and `eoc@demo.crisisgrid.app` in the other. Submit a report as the citizen and watch it appear live on the EOC map with an AI summary, without refreshing either page.

3 Complete Feature Reference

Every feature below is live in the app — none are mocked or hidden behind demo mode.

ALL ROLES

Authentication & role-based access

What it does: Sign up, log in, and stay logged in across page reloads — while every dashboard and API endpoint enforces who's allowed to do what.

HOW IT'S BUILT

JWT access tokens (15-minute expiry) held in React memory (never localStorage), paired with an httpOnly refresh-token cookie. An axios interceptor silently refreshes an expired access token and retries the request — the user never notices. Express middleware (`protect`, `authorize(...roles)`) gates every route server-side; the frontend's `ProtectedRoute` mirrors it for UX, but the real enforcement is always server-side.

CITIZEN

SOS & incident reporting

What it does: Citizens submit an SOS or a standard report with title, description, category, and location.

HOW IT'S BUILT

A single `Incident` MongoDB collection (GeoJSON location field with a 2dsphere index) covers SOS, reports, missing-persons, and resource requests via a `type` discriminator. On submit, the report is run through AI classification and duplicate detection in parallel before the response returns.

ALL ROLES

AI incident summaries ★★★★★

What it does: Instead of showing the citizen's full paragraph, every report gets a terse 1–2 sentence briefing. Example — a report describing a dam overflow with "40 people trapped" becomes: "*Flood reported. Approximately 40 people affected. Immediate evacuation recommended.*"

HOW IT'S BUILT

A keyword/pattern classifier extracts category and severity, pulls an affected-person count via regex if mentioned, and assembles a short templated sentence with an action phrase chosen by severity. When a Gemini API key is configured, the same job is delegated to Gemini with a strict-JSON prompt capped at 20 words; on any failure it falls back to the offline classifier automatically.

ALL ROLES

AI resource recommendation ★★★★★

What it does: Every incident shows a dispatch checklist, e.g. ✓ 3 Ambulances, ✓ 2 Rescue boats, ✓ Police unit, ✓ Temporary shelter — Estimated response: 15 minutes.

HOW IT'S BUILT

Deliberately a **deterministic rule table** keyed by (category, severity) — not an LLM guess. An LLM is well-suited to classifying free text into a category/severity; it's a poor fit for inventing a safety-critical resource count, since a model can produce a confident, wrong number. The AI does the classification; a transparent, auditable table converts that into a dispatch plan.

ALL ROLES

Bottom-right assistant chatbot ★★★★★

What it does: A floating chat widget available everywhere. Citizens type things like "Water is entering my house" and get safety guidance plus the nearest shelter with live distance. EOC types "Show all unverified flood reports" and gets a real, clickable, filtered list.

HOW IT'S BUILT

One shared endpoint (`POST /api/ai/chat`) branches by role server-side. For citizens: a keyword-matched advice table plus a MongoDB `$near` geo-query for the nearest open shelter (distance computed via a haversine helper). For EOC: a natural-language parser maps phrases and synonyms ("water" → flood, "unverified" → pending trust state, "active" → not resolved/rejected) onto a MongoDB filter object, executed live against the Incident collection.

ALL ROLES

Multi-layer report verification & trust score

What it does: Every report carries a live 0–100 trust score and a state — Pending, Suspicious, Verified, or Highly Trusted — so fake, mistaken, or unverified reports never look the same as a corroborated one. Anyone can crowd-vote confirm/dispute; EOC/rescue can authority-verify or reject a report outright as fake.

HOW IT'S BUILT

A weighted formula combines four independent signals: GPS consistency 30% (haversine distance between the reporter's device location and the claimed incident location, scored 100 at ≤150m decaying to 0 at 5km), Crowd votes 25% (net confirm/dispute ratio), Authority sign-off 25% (binary EOC/rescue decision), and AI text confidence 20%. A missing signal (e.g. denied GPS permission) returns a neutral score rather than being penalized. Recomputed by one pure function (`computeTrustScore`) from three call sites — report creation, a new crowd vote, or an authority decision — so the score can never drift out of sync with its inputs.

ALL ROLES

Smart calamity map

What it does: Each incident renders as an icon matching its actual type (🌊 flood, 🔥 fire, 🚑 medical, 🏚 earthquake, 🏗 structural), colored by risk level (green/yellow/orange/red — Low/Medium/High/Critical). Critical incidents pulse. Once resolved or rejected, a marker fades to grayscale in place instead of disappearing, so the map visually "clears" while staying auditable.

HOW IT'S BUILT

Leaflet + OpenStreetMap (free, no API key) with custom `divIcon` markers built per incident from its category/severity/status. The pulse is a CSS keyframe animation reused from the severity badge component; the fade is a conditional opacity + grayscale filter. Marker state updates live via Socket.IO — no page refresh needed to see a marker change color or fade.

NON-CITIZEN ROLES

Organization verification documents

What it does: Hospital, shelter, rescue team, NGO, and volunteer signups upload an ID/registration image. In a real deployment a government official would review it; here, an EOC/admin account plays that role and approves or rejects it from the directory panel.

HOW IT'S BUILT

The image is read client-side via the File API and converted to a base64 data URI, then stored directly on the user's document — no paid file-storage service required at this scale. A `status` field (not_submitted / pending / approved / rejected) tracks the review state.

EOC

Emergency contact directory

What it does: One screen listing every hospital, shelter, rescue team, and NGO with phone number, organization name, and — for hospitals/shelters — live capacity, so dispatch never has to dig through user records mid-crisis.

HOW IT'S BUILT

A dedicated endpoint joins the User collection (filtered by responder-type roles) with each user's managed Facility record in one response, sorted by role.

EOC

Broadcast alerts (top-bar notifications)

What it does: EOC can push a high-risk notification (e.g. a flash-flood warning) to the top of every connected user's screen instantly, and retract it once the risk has passed.

HOW IT'S BUILT

A small `Alert` collection plus a Socket.IO broadcast (`alert:new` / `alert:dismissed`) — active alerts are also fetched on page load so a fresh page load still shows one that was missed.

ALL ROLES

Real-time updates everywhere

What it does: New reports, status changes, verification updates, facility capacity changes, and alerts all appear instantly on every open dashboard — no polling, no manual refresh.

HOW IT'S BUILT

Socket.IO shares the same HTTP server as the Express API. Rather than hand-rolling optimistic cache patches from socket payloads, the frontend simply invalidates the relevant React Query cache keys on any relevant socket event, letting React Query re-fetch — simpler and far less race-condition-prone than manual state splicing.

EOC / RESCUE

Duplicate report detection

What it does: Two citizens reporting the same flooded street a minute apart get automatically merged instead of triggering a duplicate dispatch.

HOW IT'S BUILT

No paid vector database. A MongoDB `$near` geo-query narrows candidates to within 500m, then a from-scratch Jaccard similarity score (word-set overlap) on the title+description flags likely duplicates above a threshold.

EOC

Automatic escalation

What it does: A critical, non-suspicious incident that hasn't been dispatched yet surfaces prominently on the EOC dashboard automatically, so it can't get lost in a long queue.

HOW IT'S BUILT

A simple rule (`shouldEscalate`) evaluated after every trust-score recompute: severity is critical, trust state isn't suspicious, and status is still reported/acknowledged.

4 Technology Stack

LAYER	CHOICE	WHY
Frontend	React 18 + Vite, Tailwind CSS, React Router, TanStack Query	Fast dev server, utility-first styling, server-state caching without Redux boilerplate
Backend	Node.js + Express	The "E" and "N" of MERN; simple, well-understood REST + middleware model
Database	MongoDB (Atlas free tier), Mongoose ODM	Flexible schema for varied incident/report shapes; native GeoJSON + 2dsphere geo queries
Auth	JWT access tokens + httpOnly refresh cookie	Stateless auth without storing tokens in XSS-exposed localStorage
Realtime	Socket.IO	Reliable WebSocket abstraction with automatic fallback
Maps	OpenStreetMap tiles + Leaflet / react-leaflet	Free, no API key, no usage limits
AI	Google Gemini free tier, with a deterministic offline fallback	Zero-cost demoability; never blocked by API quota or missing keys
Deployment	Vercel (frontend), Render/Railway (backend), MongoDB Atlas (DB) — or Docker Compose locally	All free tiers; see the deployment guide for exact steps

5 System Architecture Notes

- **One User collection, one role enum** rather than 8 separate collections — a single auth identity, many possible roles, simpler JWT logic. Role-specific data (e.g. a hospital's bed count) lives on a separate `Facility` document instead.
- **Access token in memory, refresh token in an httpOnly cookie** — avoids ever putting a JWT in localStorage. An axios response interceptor catches a 401, silently refreshes, and retries once.
- **AI service is a swappable strategy** — `analyzeIncident()` tries Gemini only when a key is configured and demo mode is off, and always falls back to the deterministic classifier on any failure (missing key, network error, bad JSON, rate limit).
- **Resource recommendations and the EOC chatbot's query parser are deliberately rule-based, not LLM outputs** — an explainable, auditable table/parser rather than a black box, especially where the output could inform a safety-critical dispatch decision.
- **Verification data lives on the Incident document itself** and is recomputed by one pure function from three call sites (create, crowd vote, authority action), so the trust score can never drift out of sync with its inputs.
- **Realtime without over-engineering** — the frontend invalidates React Query cache keys on any socket event rather than hand-patching state, trading one extra network round-trip for far fewer race-condition bugs.

6 Roadmap — What's Next

Deliberately deferred to ship a smaller, fully-working v1 rather than a larger, half-working one:

- **Volunteer, NGO, and Admin dashboards** — the role system already supports them; the Hospital/Shelter portals are a working template to extend from.
- **AI image verification** (detecting flood/fire/damage in an uploaded photo, EXIF cross-check) — the trust-score architecture already has a slot for it; needs real file-upload infrastructure first.
- **Dedicated police/fire/municipal authority roles** — currently EOC/rescue stand in for "authority verification."
- **Offline-first PWA** (service worker, local report queue, background sync via IndexedDB).
- **BullMQ + Redis** background job queue — relevant once report volume is high enough that AI calls shouldn't block the request/response cycle.
- **Ollama local-model fallback** for a fully offline AI mode.
- **SMS broadcast alerts** — needs a paid provider (Twilio-class); email via Nodemailer/SMTP would be a free addition.
- **Weather/GIS overlays and AI route optimization** — free weather APIs exist, but real routing/traffic-avoidance needs a dedicated routing engine (OSRM/Mapbox).
- **Donations/funding module** with a mock payment flow.
- **Additional demo scenarios** (cyclone, earthquake, wildfire) — the seed script is structured so each is just another seed file.

7 Appendix — Demo Credentials & Quick Start

Demo accounts (password for all: `Demo@1234`)

ROLE	EMAIL
Citizen	`citizen@demo.crisisgrid.app`
EOC Control	`eoc@demo.crisisgrid.app`
Rescue Team	`rescue@demo.crisisgrid.app`
Hospital	`hospital@demo.crisisgrid.app`
Shelter	`shelter@demo.crisisgrid.app`

Quick start (Docker)

1. Copy `backend/.env.example` to `backend/.env` and fill in two JWT secrets.
2. Run `docker compose up --build` from the project root.
3. Seed demo data: `docker compose exec backend npm run seed`.
4. Open `http://localhost:5173`.

Full setup (with and without Docker) and free-tier deployment steps are documented in the project's `README.md` and the accompanying deployment guide.